

Guardrailed Text-to-SQL Analytics Platform

Complete Project Documentation

A full project guide written in simple English so that even a beginner can read, understand, and run this project from scratch.

- AST-Based SQL Compiler Firewall (SQLGlot)
- EXPLAIN Query Cost Pre-Check
- Dynamic PII Masking (SHA-256)
- Dual Database Engine (PostgreSQL + SQLite)
- AI-Powered NLP (Google Gemini + LangChain)
- Reactive Web Dashboard (Streamlit + Plotly)

Generated on 2026-10-01

Guardrailed Text-to-SQL Analytics Platform -- Complete Documentation

A full project guide written in simple English so that even a beginner can read, understand, and run this project from scratch.

Table of Contents

#	Section
1	What Is This Project?
2	How It Works (Simple Explanation)
3	Technology Stack
4	Project Folder Structure
5	Installation from Scratch (Step-by-Step)
6	How to Run the Project
7	Database Schema (SQL Tables)
8	File-by-File Explanation
9	Architecture Diagram
10	Security Guardrails -- How They Protect the Database
11	API Endpoints (FastAPI)
12	How to Test the Project
13	Sample Queries to Try on the Dashboard
14	Environment Variables Explained
15	Troubleshooting Common Errors
16	SQL Queries Used Inside the Project

1. What Is This Project?

This is a Text-to-SQL Analytics Platform. It lets users type a question in plain English (like "Show total revenue for 2026") and the system:

1. Converts the English question into a SQL query using AI (Google Gemini) or a built-in rule engine
2. Checks the SQL for safety using an AST (Abstract Syntax Tree) Compiler Firewall -- it blocks harmful queries like DROP TABLE, DELETE, etc.
3. Checks the cost of the query before running it -- prevents heavy queries from crashing the database
4. Runs the safe SQL on the database (PostgreSQL or SQLite)
5. Masks private data (emails, phone numbers) using SHA-256 hashing before showing results
6. Shows results in a beautiful web dashboard with charts, KPI cards, and a data table

In short: You type English -> system creates SQL -> checks it for safety -> runs it -> shows results with charts.

2. How It Works (Simple Explanation)

Here is the step-by-step flow of what happens when you type a question:

```
Step 1: You type a question in the web dashboard
Example: "Show total revenue and monthly sales trend for 2026"
|
v
Step 2: The system sends this question to the FastAPI backend
|
v
Step 3: The NLP Engine (LangChain + Gemini AI) converts your English into a SQL query
Example SQL: SELECT SUBSTR(OrderDate, 1, 7) AS Month,
SUM("Line Total") AS "Total Revenue"
FROM sales_order
WHERE OrderDate >= '2026-01-01'
GROUP BY Month
|
v
Step 4: The AST Compiler Firewall checks the SQL
Is it a SELECT?          --> PASS
Is it a DROP/DELETE/UPDATE? --> BLOCKED immediately
|
v
Step 5: The EXPLAIN Cost Checker runs the SQL plan
Cost is within limits?    --> PASS
Cost too high (Cartesian)? --> BLOCKED
|
v
Step 6: The database runs the safe SQL query
Returns raw data as a table
|
v
Step 7: PII Masker scans the result for sensitive columns (email, phone)
Replaces values with SHA-256 hashes like "SHA256:4a2b9c3d1e..."
|
v
Step 8: Analytics Engine computes:
- 3 KPI summary cards (Total Records, Revenue, Security Status)
- Auto-generated chart (Line chart for trends, Bar chart for categories)
|
v
Step 9: Dashboard displays everything beautifully
- KPI Cards
- Plotly Chart
- Paginated Data Table
- CSV/JSON Export buttons
```

3. Technology Stack

Component	Technology	Purpose
Backend API	Python + FastAPI	REST API server that processes queries
Frontend Dashboard	Streamlit + Plotly	Web UI where users interact
AI/NLP Engine	LangChain + Google Gemini	Converts English to SQL
SQL Firewall	SQLGlot	Parses SQL into AST and blocks dangerous queries
Primary Database	PostgreSQL 16	Enterprise database for production
Fallback Database	SQLite	Works without any setup, zero configuration
PII Masking	Python hashlib (SHA-256)	Hides sensitive personal data
Data Processing	Pandas	Handles tabular data manipulation
Charts	Plotly	Creates interactive line and bar charts
Environment Config	python-dotenv	Loads settings from .env file

4. Project Folder Structure

```
text-to-sql/                                <-- Root project folder
|
|-- .devcontainer/
|   +-- devcontainer.json                   <-- Config for GitHub Codespaces / VS Code containers
|
|-- .env                                    <-- Your secret settings (API keys, DB passwords)
|-- .env.example                           <-- Template showing what settings are needed
|-- .gitignore                             <-- Tells Git which files to ignore
|
|-- .streamlit/
|   +-- config.toml                         <-- Streamlit theme and server settings
|
|-- backend/                               <-- All server-side Python code
|   |-- __init__.py                       <-- Makes "backend" a Python package
|   |-- main.py                           <-- FastAPI app - all API routes live here
|   |-- analytics.py                       <-- KPI card calculator + chart generator
|   |
|   |-- database/                          <-- Database connection and setup
|   |   |-- __init__.py                   <-- Makes "database" a Python package
|   |   |-- connection.py                 <-- Connects to PostgreSQL or SQLite
|   |   +-- db_setup.py                   <-- Creates tables, loads CSV data, creates indexes
|   |
|   |-- nlp/                               <-- Natural Language Processing
|   |   |-- __init__.py                   <-- Makes "nlp" a Python package
|   |   +-- llm_engine.py                 <-- Gemini AI integration + fallback SQL synthesizer
|   |
|   +-- security/                          <-- All security guardrails
|       |-- __init__.py                   <-- Makes "security" a Python package
|       |-- ast_guardrail.py               <-- AST Compiler Firewall (blocks DROP/DELETE/etc.)
|       |-- cost_checker.py                <-- EXPLAIN query cost pre-check
|       +-- pii_masker.py                  <-- SHA-256 PII data masking
|
|-- data/
|   |-- sales_data.db                      <-- SQLite database file (auto-created)
|   |-- sample_retail_inventory_2026.csv   <-- Sample CSV data
```

```
| +-- sample_retail_inventory_2026.xlsx <-- Sample Excel data
|
|-- frontend/                                <-- All UI/dashboard code
| |-- app.py                                <-- Main Streamlit dashboard (2000+ lines)
| +-- saas_components.py                    <-- Reusable UI components, SVG icons, data grids
|
|-- tests/                                  <-- Automated test files
| |-- test_guardrail.py                     <-- Tests for AST Firewall + PII Masking
| +-- test_pipeline.py                     <-- End-to-end pipeline integration test
|
|-- logs/                                  <-- Runtime log files (auto-created)
|-- requirements.txt                        <-- List of Python packages to install
|-- run.py                                <-- Master launcher script (starts backend + frontend)
|-- streamlit_app.py                       <-- Entry point for Streamlit Community Cloud
|-- implementation_plan.md                 <-- Detailed project plan document
+-- README.md                             <-- Quick-start guide
```

5. Installation from Scratch (Step-by-Step)

Prerequisites (What You Need First)

Before you start, make sure you have these installed on your computer:

Software	Version	How to Check	Download Link
Python	3.10 or higher	Open terminal, type: <code>python --version</code>	python.org/downloads
pip	Latest	Type: <code>pip --version</code>	Comes with Python
Git	Any version	Type: <code>git --version</code>	git-scm.com

NOTE

PostgreSQL is optional. This project works perfectly fine with just SQLite (which needs zero installation). PostgreSQL is only needed if you want the enterprise-grade database setup.

Step 1: Download the Project

Open your terminal (Command Prompt / PowerShell / VS Code Terminal) and run:

```
BASH
cd C:\Users\MSI\Desktop
git clone <your-repo-url> text-to-sql
cd text-to-sql
```

Or if you already have the folder, just navigate to it:

```
BASH
cd C:\Users\MSI\Desktop\text-to-sql
```

Step 2: Create a Virtual Environment (Recommended)

A virtual environment keeps this project's packages separate from your other Python projects.

On Windows (PowerShell):

POWERSHELL

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

On Windows (Command Prompt):

CMD

```
python -m venv venv
venv\Scripts\activate.bat
```

TIP

After activation, you will see (venv) at the beginning of your terminal line. This means it is working.

WARNING

PowerShell Error? If you get "running scripts is disabled", run this first: ``powershell Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser``

Step 3: Install All Required Python Packages

BASH

```
pip install -r requirements.txt
```

This installs all 15 packages listed in requirements.txt:

Package	What It Does
fastapi	Creates the REST API backend
uvicorn	Runs the FastAPI server
streamlit	Creates the web dashboard
plotly	Makes interactive charts
pandas	Works with tabular data
psycopg2-binary	Connects Python to PostgreSQL
sqlalchemy	Database toolkit (helps load data into PostgreSQL)
sqlglot	Parses SQL into AST trees (for the security firewall)
langchain	AI orchestration framework
langchain-google-genai	Google Gemini connector for LangChain
google-genai	Google AI SDK
python-dotenv	Reads settings from .env file
requests	Makes HTTP calls (frontend calls backend)
pydantic	Data validation for API requests/responses
openpyxl	Reads Excel .xlsx files

Step 4: Set Up the Environment File

The .env file stores your configuration. A template is already provided:

BASH

```
copy .env.example .env
```

Now open the .env file and edit it:

ENV

```
# Google Gemini API Key (Optional - project works without it)
GOOGLE_API_KEY=your_gemini_api_key_here

# PostgreSQL Config (Optional - only if PostgreSQL is installed)
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=text_to_sql_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres

# Read-Only Role (auto-created by setup script)
POSTGRES_READONLY_USER=readonly_analyst
POSTGRES_READONLY_PASSWORD=readonly_secure_pass_2026

# SQLite Fallback (Always works, no setup needed)
SQLITE_DB_PATH=data/sales_data.db

# System Settings
MAX_QUERY_ROWS=1000
MAX_QUERY_COST=100000.0
PII_MASKING_ENABLED=true
FASTAPI_PORT=8000
STREAMLIT_PORT=8501
```

IMPORTANT

No Gemini API Key? No problem! Leave `GOOGLE_API_KEY=` empty or delete the line entirely. The project has a built-in SQL synthesizer that works offline without any API key. You can still test all features.

TIP

To get a free Gemini API key: Go to aistudio.google.com then Sign in with Google then Click "Get API Key" then Copy the key and paste it in `.env`.

Step 5: Set Up the Database

The project needs CSV data files to populate the database. These CSV files should be located at:

```
C:\Users\MSI\Desktop\Text-to-SQL-Chatbot-main\Data_CSV\
```

Expected CSV files in that folder:

- 2017_Budgets.csv
- Customers.csv
- Products.csv
- Regions.csv
- sales_order.csv
- State_Regions.csv

Now run the database setup:

BASH

```
python -m backend.database.db_setup
```

What this command does:

1. Reads all 6 CSV files

2. Adds fake email and phone columns to the Customers table (to demonstrate PII masking)
3. Generates 1,420 synthetic records for the year 2026 (to match the wireframe demo)
4. Loads everything into SQLite (data/sales_data.db)
5. If PostgreSQL is installed and running, also loads into PostgreSQL with:
 - B-Tree indexes for fast queries
 - A restricted read-only user (readonly_analyst) for security

Expected output:

```
=====
Guardrailed Text-to-SQL Engine - Database Setup & Ingestion
=====
Reading CSV files from: C:\Users\MSI\Desktop\Text-to-SQL-Chatbot-main\Data_CSV

--- Loading datasets into SQLite: data/sales_data.db ---
[SQLite] Table 'budgets_2017': 30 rows loaded.
[SQLite] Table 'customers': 175 rows loaded.
[SQLite] Table 'products': 30 rows loaded.
[SQLite] Table 'regions': 994 rows loaded.
[SQLite] Table 'sales_order': 65,524 rows loaded.
[SQLite] Table 'state_regions': 48 rows loaded.
[SQLite] B-Tree indexes created successfully.

--- Loading datasets into PostgreSQL: localhost:5432/text_to_sql_db ---
[PostgreSQL] Table 'budgets_2017': 30 rows loaded.
...

Database initialization complete.
```

NOTE

If PostgreSQL is not installed, you will see a notice saying "Could not configure PostgreSQL" - that is perfectly fine! SQLite is ready and will be used automatically.

Step 6: Run the Project

BASH

```
python run.py
```

What happens:

1. Checks if the SQLite database exists; if not, runs the setup automatically
2. Starts the FastAPI backend on <http://localhost:8000>
3. Starts the Streamlit dashboard on <http://localhost:8501>

Expected output:

```
=====
[START] Starting Guardrailed Text-to-SQL Analytics Platform
=====

[1/2] Launching FastAPI Backend on http://localhost:8000 ...
[2/2] Launching Streamlit Web Dashboard on http://localhost:8501 ...

=====
[ONLINE] Guardrailed Text-to-SQL Platform is LIVE!
```

```
-> Dashboard UI: http://localhost:8501
-> FastAPI Docs: http://localhost:8000/docs
-> System Health: http://localhost:8000/api/health
=====
Press Ctrl+C to terminate all services.
```

Now open your browser and go to: <http://localhost:8501>

TIP

To stop the project, press Ctrl+C in the terminal.

6. How to Run the Project

Option A: Run Everything Together (Recommended)

```
BASH
python run.py
```

This starts both the backend and frontend at once.

Option B: Run Backend and Frontend Separately

Terminal 1 - Backend:

```
BASH
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

Terminal 2 - Frontend:

```
BASH
python -m streamlit run frontend/app.py --server.port 8501 --server.headless true
```

Option C: Run Only the Backend API

```
BASH
python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
```

Then test the API at: <http://localhost:8000/docs> (Swagger UI)

7. Database Schema (SQL Tables)

The project uses 6 tables. Here is the complete schema:

Table 1: sales_order (65,524 rows)

This is the main table. It contains all sales transactions.

```
SQL
CREATE TABLE sales_order (
  OrderNumber      TEXT,      -- Example: "S0-2026-00001"
  OrderDate        TEXT,      -- Example: "2026-03-15"
  "Customer Name Index" INTEGER, -- Foreign key to customers table
  Channel          TEXT,      -- "Wholesale", "Distributor", "Export"
```

```

"Currency Code"          TEXT,      -- "USD", "EUR", etc.
"Warehouse Code"         TEXT,      -- Warehouse identifier
"Delivery Region Index"  INTEGER,   -- Foreign key to regions
"Product Description Index" INTEGER, -- Foreign key to products
"Order Quantity"         INTEGER,   -- How many units ordered
"Unit Price"             REAL,      -- Price per unit
"Line Total"             REAL,      -- Total revenue for this order line
"Total Unit Cost"        REAL       -- Cost per unit
);

-- Indexes created for fast queries:
CREATE INDEX idx_sales_date    ON sales_order (OrderDate);
CREATE INDEX idx_sales_channel ON sales_order (Channel);
CREATE INDEX idx_sales_cust    ON sales_order ("Customer Name Index");
CREATE INDEX idx_sales_prod    ON sales_order ("Product Description Index");

```

Table 2: customers (175 rows)

Customer directory with PII fields for masking demonstration.

SQL

```

CREATE TABLE customers (
  "Customer Index"  INTEGER,      -- Primary key
  "Customer Names"  TEXT,         -- Company name, e.g. "Medline"
  Email             TEXT,         -- Auto-generated, e.g. "medline@corp.net"
  "Phone Number"    TEXT         -- Auto-generated, e.g. "+1-555-123-4567"
);

CREATE INDEX idx_cust_id ON customers ("Customer Index");

```

NOTE

The Email and Phone Number columns are fake (auto-generated). They exist to demonstrate the PII masking feature. These columns get hashed with SHA-256 when results are shown.

Table 3: products (30 rows)

Product catalog.

SQL

```

CREATE TABLE products (
  "Index"          INTEGER,      -- Product ID
  "Product Name"   TEXT         -- e.g. "Product 1", "Product 12"
);

```

Table 4: budgets_2017 (30 rows)

Budget allocations per product for the year 2017.

SQL

```

CREATE TABLE budgets_2017 (
  "Product Name"   TEXT,         -- e.g. "Product 1"
  "2017 Budgets"   REAL         -- Budget amount, e.g. 12500.00
);

```

Table 5: regions (994 rows)

Geographic and demographic data for US cities.

SQL

```
CREATE TABLE regions (
  id            INTEGER,      -- Primary key
  name          TEXT,         -- City name, e.g. "New York"
  county        TEXT,         -- County name
  state_code    TEXT,         -- e.g. "NY"
  state         TEXT,         -- e.g. "New York"
  type          TEXT,         -- e.g. "City", "Town"
  latitude      REAL,         -- GPS latitude
  longitude     REAL,         -- GPS longitude
  area_code     INTEGER,      -- Phone area code
  population    INTEGER,      -- City population
  households    INTEGER,      -- Number of households
  median_income INTEGER,      -- Median household income
  land_area     INTEGER,      -- Land area (sq miles)
  water_area    INTEGER,      -- Water area (sq miles)
  time_zone     TEXT         -- e.g. "America/New_York"
);
```

Table 6: state_regions (48 rows)

Maps US states to their geographic region.

SQL

```
CREATE TABLE state_regions (
  "State Code" TEXT,         -- e.g. "CA"
  State        TEXT,         -- e.g. "California"
  Region       TEXT         -- e.g. "West", "South", "Northeast"
);
```

Entity Relationship Summary

```
customers."Customer Index" <--> sales_order."Customer Name Index"
products."Index"           <--> sales_order."Product Description Index"
regions.id                 <--> sales_order."Delivery Region Index"
state_regions."State Code" <--> regions.state_code
budgets_2017."Product Name" <--> products."Product Name"
```

8. File-by-File Explanation

Root Files

run.py

Purpose: Master launcher. Starts both services with one command.

What it does:

1. Checks if the SQLite database exists. If not, runs db_setup.py automatically
2. Starts FastAPI backend on port 8000 as a background process
3. Starts Streamlit frontend on port 8501 as a background process

4. Monitors both processes and terminates them when you press Ctrl+C

streamlit_app.py

Purpose: Entry point for Streamlit Community Cloud deployment. Simply loads and runs frontend/app.py.

requirements.txt

Purpose: Lists all Python packages the project depends on. pip install -r requirements.txt installs them all.

.env

Purpose: Configuration file storing API keys, database credentials, and system settings. This file is not committed to Git (listed in .gitignore).

Backend Files

backend/main.py

Purpose: The heart of the backend. This is the FastAPI application with all API endpoints.

Key endpoints defined here:

- POST /api/query - Main pipeline: takes a question, returns SQL + results + charts
- GET /api/health - Checks if databases and services are running
- GET /api/schema - Returns all table names, columns, and row counts
- GET /api/audit - Returns security audit statistics (total queries, blocked attacks)

Full pipeline executed in /api/query:

1. Extract database schema metadata
2. Generate SQL from English (via Gemini AI or built-in synthesizer)
3. Run AST Compiler Firewall check
4. Run EXPLAIN Cost pre-check
5. Execute the SQL query on the database
6. Apply PII masking to sensitive columns
7. Extract KPI card metrics
8. Generate chart specification
9. Generate plain English explanation
10. Return everything as a JSON response

backend/analytics.py

Purpose: Computes the 3 KPI summary cards and auto-generates chart configurations.

Key functions:

- extract_kpi_cards() - Calculates Total Records, Primary Aggregate (revenue/sum), Security Status
- auto_generate_chart() - Detects if data has dates then Line Chart, or categories then Bar Chart
- explain_query_plain_english() - Converts SQL back into step-by-step English explanation
- generate_executive_insights() - Creates business insights like "Peak revenue in March"

- `generate_direct_answer()` - Generates a one-line natural language answer
-

Database Files

backend/database/db_setup.py

Purpose: One-time setup script. Creates all tables and loads data.

What it does:

1. Finds the CSV data directory
2. Reads all 6 CSV files using Pandas
3. Adds fake email/phone columns to the Customers table
4. Generates 1,420 synthetic 2026 records (to match the wireframe demo)
5. Loads everything into SQLite (always)
6. Loads everything into PostgreSQL (if available)
7. Creates B-Tree indexes for fast queries
8. Creates a read-only PostgreSQL user (`readonly_analyst`)

backend/database/connection.py

Purpose: Handles all database connections and query execution.

Key functions:

- `get_sqlite_connection()` - Opens a connection to the SQLite database file
- `get_postgres_connection()` - Opens a connection to PostgreSQL (prefers read-only user)
- `check_health()` - Tests if PostgreSQL and SQLite are accessible
- `get_schema_metadata()` - Extracts table names, column names, data types, and row counts
- `execute_query()` - Runs a SQL query and returns results as a Pandas DataFrame

Auto-fallback logic: If PostgreSQL is not available, the system automatically uses SQLite. No configuration change needed.

Security Files

backend/security/ast_guardrail.py

Purpose: The SQL Compiler Firewall. This is the core security feature.

How it works:

1. Keyword scan - First pass: checks for forbidden words (DROP, DELETE, UPDATE, INSERT, ALTER, TRUNCATE, CREATE, GRANT, REVOKE, EXEC, EXECUTE, SHUTDOWN)
2. AST parsing - Uses `sqlglot` to parse the SQL into a syntax tree
3. Root node check - The root node must be a SELECT or UNION (read-only)
4. Recursive tree walk - Walks every node in the AST tree looking for forbidden node types
5. Multi-statement check - Blocks semicolon injection (e.g., `SELECT 1; DROP TABLE x`)
6. Clean SQL regeneration - If safe, regenerates clean formatted SQL from the AST

Return values:

- is_safe: true + status: "[AST Read-Only OK]" means query is safe
- is_safe: false + status: "[BLOCKED: ...]" means query is dangerous

Blocked AST node types: Drop, Delete, Update, Insert, Alter, Create, TruncateTable, Command, Set, Kill, Transaction, Commit, Rollback

backend/security/cost_checker.py

Purpose: Prevents resource-heavy queries from running.

How it works:

- PostgreSQL: Runs EXPLAIN (FORMAT JSON) to get the query plan, estimated cost, and row count. If cost exceeds the limit (MAX_QUERY_COST, default 100,000), the query is blocked.
- SQLite: Runs EXPLAIN QUERY PLAN to check for unindexed scans and Cartesian joins.

backend/security/pii_masker.py

Purpose: Automatically finds and masks personal/sensitive data in query results.

How it works:

1. Scans all column names in the result
2. If a column name matches any of these patterns, it is a PII column:
 - email, phone, contact, mobile, ssn, social_security, credit_card, card_number, tax_id, password, secret
3. Every value in that column is replaced with a SHA-256 hash:
 - Original: alice@corp.com
 - Masked: SHA256:4a2b9c3d1e...

Important: The hashing is deterministic. The same input always produces the same hash. This preserves data relationships without exposing the actual data.

NLP / AI Files

backend/nlp/llm_engine.py

Purpose: Converts natural language questions into SQL queries.

Two modes of operation:

Mode 1: Gemini AI (when API key is provided)

- Uses langchain-google-genai to connect to Google Gemini 1.5 Flash model
- Sends the database schema + user question as a prompt
- The AI returns a SQL query
- The system cleans the SQL (removes markdown formatting, extracts just the query)

Mode 2: Built-in Synthesizer (when no API key)

- A rule-based pattern matcher that handles common question types:
 - "Show total revenue for 2026" generates the correct GROUP BY month query
 - "Total customers" generates SELECT COUNT(*) FROM customers
 - "Sales by channel" generates the channel breakdown query

- And many more patterns

Key functions:

- `generate_sql_query()` - Main function. Tries Gemini first, falls back to synthesizer
 - `sanitize_sql()` - Strips markdown code fences and extracts raw SQL
 - `synthesize_fallback_sql()` - Pattern-matching engine for offline operation
-

Frontend Files

frontend/app.py

Purpose: The main web dashboard. This is what users see and interact with.

Dashboard layout (top to bottom):

1. Navbar - Brand logo, database engine indicator, PII masking status
2. Search Bar - Text input where you type your English question + Execute button
3. Quick Prompt Buttons - Pre-built example questions you can click
4. 3 KPI Cards - Total Records | Primary Aggregate | Security Status
5. Plotly Chart - Auto-generated line chart (for trends) or bar chart (for categories)
6. Data Table - Paginated grid showing query results with PII masking applied
7. Export Buttons - Download results as CSV or JSON
8. Security Inspector - Shows generated SQL, AST node tree, EXPLAIN cost plan
9. Database Schema Viewer - Shows all tables, columns, and row counts
10. Sidebar - API key input, database engine toggle, PII masking toggle, audit stats

frontend/saas_components.py

Purpose: Reusable UI components. Contains:

- 25+ custom SVG icons (Shield, Database, Chart, Lock, etc.)
 - `render_saas_navbar()` - Top navigation bar
 - `render_kpi_card()` - KPI metric card
 - `render_saas_data_grid()` - Styled data table with column type badges
 - `render_schema_table()` - Database schema display
 - `render_security_benchmark_table()` - Red-team security test matrix
 - `render_ast_nodes_table()` - AST tree node viewer
 - `render_executive_insights()` - Business insights panel
-

Test Files

tests/test_guardrail.py

Tests 3 things:

1. Safe queries pass the firewall:
 - `SELECT * FROM customers` passes with [AST Read-Only OK]

- SELECT COUNT(*) FROM sales_order WHERE ... passes
 - SELECT ... GROUP BY ... passes
 - WITH ... SELECT ... (CTE) passes
2. Dangerous queries are blocked:
- DROP TABLE customers - BLOCKED
 - DELETE FROM sales_order - BLOCKED
 - UPDATE customers SET ... - BLOCKED
 - INSERT INTO customers VALUES(...) - BLOCKED
 - ALTER TABLE products ADD COLUMN ... - BLOCKED
 - TRUNCATE TABLE sales_order - BLOCKED
 - CREATE TABLE evil (...) - BLOCKED
 - SELECT *; DROP TABLE ... (injection) - BLOCKED
3. PII masking works correctly:
- Email column gets hashed to SHA256:...
 - Phone Number column gets hashed to SHA256:...
 - Customer Names column stays untouched (not PII)

tests/test_pipeline.py

Tests the full end-to-end pipeline:

1. Generate SQL for "Show total revenue and monthly sales trend for 2026"
2. Validate it through the AST Firewall
3. Run the EXPLAIN cost pre-check
4. Execute the query on the database
5. Apply PII masking
6. Extract KPI cards and verify security status is [AST Read-Only OK]
7. Generate chart and verify it is a line chart

9. Architecture Diagram

```

[ User Question ]
|
v
+-----+
| LangChain NLP Engine |
| (Schema Reflection) |
+-----+-----+
| Raw SQL
v
+-----+
| SQLGlot AST Firewall | <-- [Blocks DROP/DELETE/ALTER/etc.]
+-----+-----+
| Validated SELECT
v
+-----+
| EXPLAIN Cost Pre-Check | <-- [Blocks Resource-Heavy Queries]
+-----+-----+

```

```

| Safe Query
v
+-----+
| PostgreSQL 16 Engine | (Dual-mode with SQLite)
| (Read-Only Role)    |
+-----+-----+
| Raw Records
v
+-----+
| Dynamic PII Masker   | <-- [SHA-256 for Emails & Phones]
+-----+-----+
| Masked Data
v
+-----+
| FastAPI Core API     | (:8000)
+-----+-----+
| JSON Payload
v
+-----+
| Streamlit Dashboard UI | (:8501)
| - 3 KPI Summary Cards |
| - Auto Plotly Chart   |
| - Paginated Data Grid |
+-----+

```

10. Security Guardrails - How They Protect the Database

Guardrail 1: AST Compiler Firewall

Attack	SQL Example	Result
Drop table	DROP TABLE customers	BLOCKED - Keyword DROP detected
Delete records	DELETE FROM sales_order WHERE 1=1	BLOCKED - Keyword DELETE detected
Update data	UPDATE customers SET name = 'Hacked'	BLOCKED - AST root is Update
Insert records	INSERT INTO customers VALUES (999, 'Evil')	BLOCKED - AST root is Insert
Alter schema	ALTER TABLE products ADD COLUMN secret TEXT	BLOCKED - Keyword ALTER detected
Truncate table	TRUNCATE TABLE sales_order	BLOCKED - AST node TruncateTable detected
SQL Injection	SELECT 1; DROP TABLE x	BLOCKED - Multiple statements detected
Safe SELECT	SELECT * FROM customers	PASSED - [AST Read-Only OK]
Safe CTE	WITH cte AS (SELECT ...) SELECT ...	PASSED - [AST Read-Only OK]

Guardrail 2: EXPLAIN Cost Pre-Check

- Runs the query plan before actual execution
- If estimated cost exceeds 100,000, the query is blocked
- Prevents Cartesian joins (accidental cross-product of tables)
- Prevents full table scans on very large tables

Guardrail 3: PII Masking

- Automatically detects columns named email, phone, contact, ssn, etc.

- Replaces real values with SHA-256 hashes
- Example: alice@corp.com becomes SHA256:4a2b9c3d1e...

Guardrail 4: Read-Only Database Role

- PostgreSQL user readonly_analyst has only SELECT permission
- Cannot create, modify, or delete any data even if a query somehow bypassed other checks
- This is enforced at the database level itself

11. API Endpoints (FastAPI)

Once the backend is running, you can see all API documentation at: <http://localhost:8000/docs>

GET /api/health

Purpose: Check if the system is running.

Example Response:

```
JSON
{
  "status": "healthy",
  "databases": {
    "postgres": false,
    "sqlite": true,
    "active_engine": "sqlite"
  },
  "firewall": "SQLGlot AST Active",
  "pii_protection": "SHA-256 Enabled"
}
```

POST /api/query

Purpose: The main endpoint. Send a natural language question and get results.

Request Body:

```
JSON
{
  "question": "Show total revenue and monthly sales trend for 2026",
  "database_type": "auto",
  "pii_masking": true,
  "api_key": null
}
```

Response (simplified):

```
JSON
{
  "success": true,
  "question": "Show total revenue and monthly sales trend for 2026",
  "sql": "SELECT SUBSTR(OrderDate, 1, 7) AS Month, COUNT(*) AS Total_Orders, ROUND(SUM(Line_Total), 2) AS Tot...",
  "security": {
    "is_safe": true,
    "status": "[AST Read-Only OK]",
  }
}
```

```
"node_types": ["Select"]
},
"cost_check": {
  "passed": true,
  "engine": "sqlite",
  "plan_summary": "SEARCH sales_order USING INDEX idx_sales_date"
},
"kpis": {
  "total_records": "12",
  "primary_aggregate": "Rs 248,500.00",
  "security_status": "[AST Read-Only OK]"
},
"chart": {
  "type": "line",
  "title": "Monthly Trend: Total Revenue over Month"
},
"table_data": [
  { "Month": "2026-01", "Total Orders": 112, "Total Revenue": 18500.00 }
],
"pii_masked_columns": [],
"active_engine": "sqlite"
}
```

GET /api/schema

Purpose: Get all table names, columns, and row counts.

Query Parameter: ?db_type=auto (or postgres or sqlite)

GET /api/audit

Purpose: Security audit statistics.

Response:

JSON

```
{
  "audit": {
    "total_queries": 15,
    "safe_queries": 12,
    "blocked_attacks": 3,
    "pii_masked_columns_count": 6,
    "recent_events": [
      { "query": "Show revenue for 2026", "status": "APPROVED", "rows": 12 },
      { "query": "DROP TABLE customers", "status": "BLOCKED" }
    ]
  }
}
```

12. How to Test the Project

Run Security Tests (AST Firewall + PII Masking)

BASH

```
python -m unittest tests.test_guardrail -v
```

Expected output:

```
test_dangerous_queries_blocked ... ok
test_pii_dynamic_masking ... ok
test_safe_queries_pass ... ok
-----
```

```
Ran 3 tests in 0.XXXs
```

```
OK
```

Run End-to-End Pipeline Test

BASH

```
python -m unittest tests.test_pipeline -v
```

Expected output:

```
test_2026_wireframe_query_flow ... ok
```

```
Query executed successfully. Result row count: 12
```

```
Month  Total Orders  Total Revenue
```

```
0  2026-01           112       18500.00
```

```
...
```

```
-----
Ran 1 test in X.XXXs
```

```
OK
```

Run All Tests Together

BASH

```
python -m unittest discover -s tests -v
```

13. Sample Queries to Try on the Dashboard

Open the dashboard at <http://localhost:8501> and try these questions:

Safe Queries (will PASS the firewall)

Question	What It Does
Show total revenue and monthly sales trend for 2026	Line chart of monthly revenue
Total number of customers	Shows customer count
Show customer directory with emails	Shows customer list (emails will be masked!)
Sales by channel	Bar chart: revenue by Wholesale/Export/Distributor
Top 10 regions by population	Shows most populated cities
Show all budgets	Shows product budget allocations
What tables are in the database	Lists all 6 database tables
Budget of Product 12	Shows specific product budget
Total number of orders	Shows total order count
Total number of products	Shows total product count
Show product list	Shows all product names

Dangerous Queries (will be BLOCKED by the firewall)

Question	What Happens
Drop table customers	BLOCKED - "Disallowed keyword DROP"
Delete all sales orders	BLOCKED - "Disallowed keyword DELETE"
Update customer names to hacked	BLOCKED - "Disallowed keyword UPDATE"

TIP

Try the dangerous queries! The security panel will turn red and show exactly how the AST Firewall blocked the attack. This is great for project demonstrations.

14. Environment Variables Explained

All settings are in the .env file:

Variable	Default Value	Purpose
GOOGLE_API_KEY	(empty)	Google Gemini API key for AI SQL generation. Leave empty for built-in..
POSTGRES_HOST	localhost	PostgreSQL server address
POSTGRES_PORT	5432	PostgreSQL port
POSTGRES_DB	text_to_sql_db	Database name
POSTGRES_USER	postgres	PostgreSQL admin username
POSTGRES_PASSWORD	postgres	PostgreSQL admin password
POSTGRES_READONLY_USERNAME	readonly_analyst	Read-only role username (auto-created)
POSTGRES_READONLY_PASSWORD	readonly_secure_pas..	Read-only role password
SQLITE_DB_PATH	data/sales_data.db	Path to the SQLite database file
MAX_QUERY_ROWS	1000	Maximum rows returned to the browser
MAX_QUERY_COST	100000.0	Maximum EXPLAIN query plan cost
PII_MASKING_ENABLED	true	Enable/disable PII column masking
FASTAPI_PORT	8000	Port for the backend API
STREAMLIT_PORT	8501	Port for the web dashboard

15. Troubleshooting Common Errors

Error: "Could not locate Data_CSV directory"

Cause: The database setup cannot find the CSV source files.

Fix: Make sure your CSV files exist at:

```
C:\Users\MSI\Desktop\Text-to-SQL-Chatbot-main\Data_CSV\
```

Or place them in a Data_CSV folder inside the project root.

Error: "SQLite database not found"

Cause: You have not run the database setup yet.

Fix:

```
BASH
python -m backend.database.db_setup
```

Error: "ModuleNotFoundError: No module named 'backend'"

Cause: You are running the script from the wrong directory.

Fix: Always run commands from the project root:

```
BASH
cd C:\Users\MSI\Desktop\text-to-sql
python run.py
```

Error: Port 8000 or 8501 is already in use

Cause: A previous instance is still running.

Fix (Windows):

```
POWERSHELL
# Find and kill the process on port 8000
netstat -ano | findstr :8000
taskkill /PID <PID_NUMBER> /F

# Find and kill the process on port 8501
netstat -ano | findstr :8501
taskkill /PID <PID_NUMBER> /F
```

Error: "pip install fails" or "Package not found"

Fix: Make sure your virtual environment is activated, then:

BASH

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Error: PowerShell script execution policy

Fix:

POWERSHELL

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Dashboard shows "Connection refused" or "Backend not responding"

Cause: The FastAPI backend is not running.

Fix: Make sure the backend is running on port 8000:

BASH

```
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

PostgreSQL connection fails

Cause: PostgreSQL is not installed or not running.

Fix: This is NOT a problem. The system automatically uses SQLite as fallback. You do not need PostgreSQL. If you want PostgreSQL:

1. Install PostgreSQL 16 from [postgresql.org](https://www.postgresql.org)
2. Make sure the service is running
3. Update the .env file with correct credentials

16. SQL Queries Used Inside the Project

Below are the actual SQL queries the built-in synthesizer generates for common questions.

Monthly Revenue Trend for 2026 (SQLite)

SQL

```
SELECT
SUBSTR(OrderDate, 1, 7) AS Month,
COUNT(*) AS "Total Orders",
ROUND(SUM("Line Total"), 2) AS "Total Revenue"
FROM sales_order
WHERE OrderDate >= '2026-01-01' AND OrderDate <= '2026-12-31'
GROUP BY SUBSTR(OrderDate, 1, 7)
ORDER BY Month ASC
```

Monthly Revenue Trend for 2026 (PostgreSQL)

SQL

```
SELECT
  TO_CHAR("OrderDate"::date, 'YYYY-MM') AS "Month",
  COUNT(*) AS "Total Orders",
  ROUND(SUM("Line Total")::numeric, 2) AS "Total Revenue"
FROM sales_order
WHERE "OrderDate" >= '2026-01-01' AND "OrderDate" <= '2026-12-31'
GROUP BY TO_CHAR("OrderDate"::date, 'YYYY-MM')
ORDER BY "Month" ASC
```

Sales by Channel (SQLite)

SQL

```
SELECT
  Channel,
  COUNT(*) AS "Total Orders",
  ROUND(SUM("Line Total"), 2) AS Revenue
FROM sales_order
GROUP BY Channel
ORDER BY Revenue DESC
```

Total Customer Count

SQL

```
SELECT COUNT(*) AS "Total Customers" FROM customers
```

Customer Directory with PII

SQL

```
SELECT "Customer Index", "Customer Names", "Email", "Phone Number"
FROM customers LIMIT 50
```

Product Budget (Product 12)

SQL

```
SELECT "Product Name", "2017 Budgets" AS "Budget"
FROM budgets_2017
WHERE "Product Name" LIKE '%Product 12%'
```

Top Regions by Population

SQL

```
SELECT name AS "City", state, population, median_income
FROM regions
ORDER BY population DESC
LIMIT 10
```

List All Database Tables (SQLite)

SQL

```
SELECT
  name AS "Table Name",
  type AS "Object Type"
FROM sqlite_master
WHERE type = 'table' AND name NOT LIKE 'sqlite_%'
ORDER BY name ASC
```

List All Database Tables (PostgreSQL)

SQL

```
SELECT
table_name AS "Table Name",
table_type AS "Table Type"
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name ASC
```

Database Setup SQL (Index Creation)

SQL

```
-- SQLite Indexes
CREATE INDEX IF NOT EXISTS idx_sales_date      ON sales_order (OrderDate);
CREATE INDEX IF NOT EXISTS idx_sales_channel  ON sales_order (Channel);
CREATE INDEX IF NOT EXISTS idx_sales_cust     ON sales_order ("Customer Name Index");
CREATE INDEX IF NOT EXISTS idx_sales_prod     ON sales_order ("Product Description Index");
CREATE INDEX IF NOT EXISTS idx_cust_id       ON customers ("Customer Index");
```

PostgreSQL Read-Only Role Setup

SQL

```
CREATE USER readonly_analyst WITH PASSWORD 'readonly_secure_pass_2026';
GRANT CONNECT ON DATABASE text_to_sql_db TO readonly_analyst;
GRANT USAGE ON SCHEMA public TO readonly_analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_analyst;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO readonly_analyst;
```

IMPORTANT

Quick Summary - Minimum Steps to Run the Project: 1. cd C:\Users\MSI\Desktop\text-to-sql 2. python -m venv venv then .\venv\Scripts\Activate.ps1 3. pip install -r requirements.txt 4. python -m backend.database.db_setup 5. python run.py 6. Open browser at <http://localhost:8501>